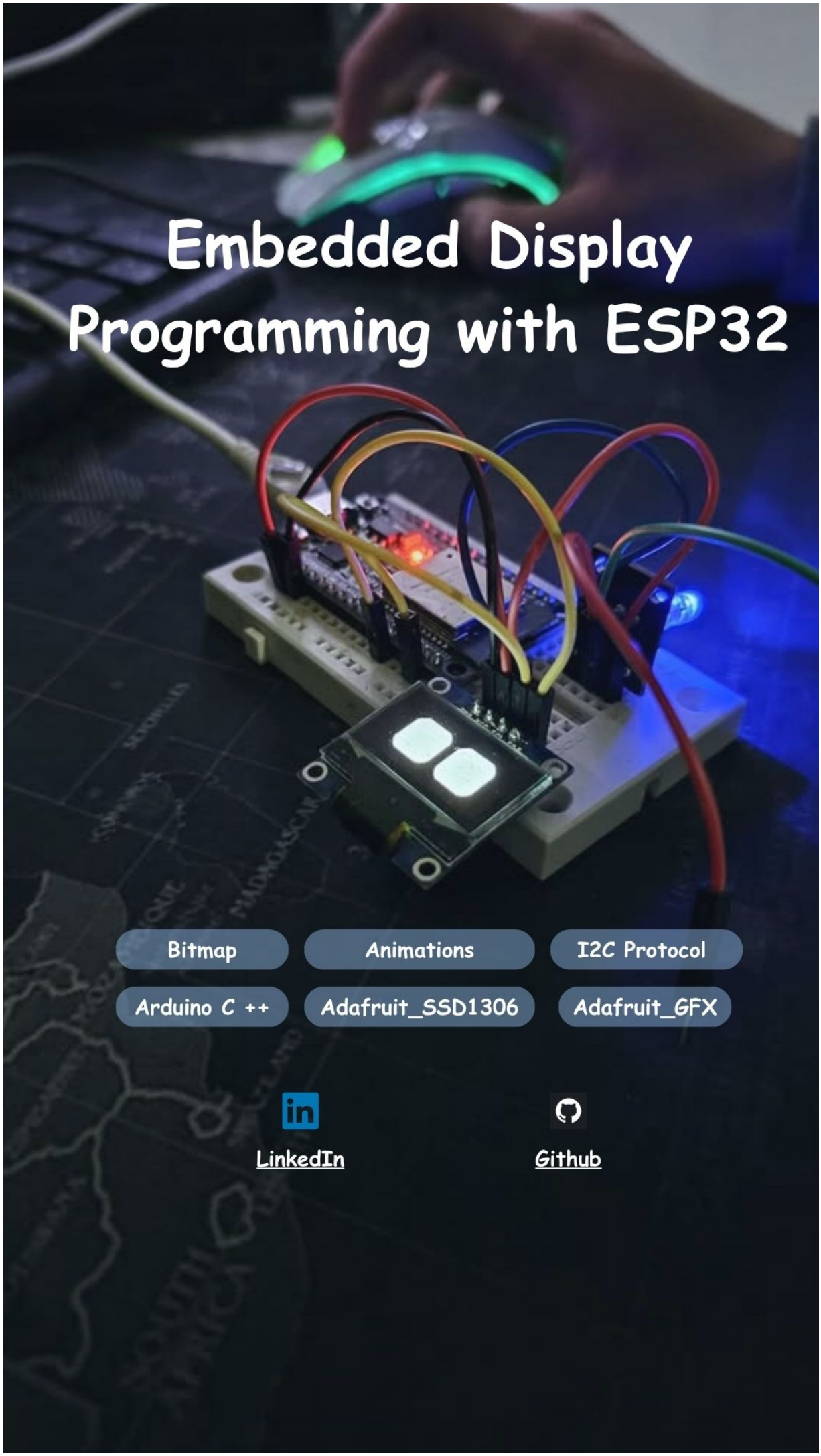

Embedded Display
Programming with ESP32
Bitmap
Animations
I2C Protocol
Arduino C ++
Adafruit_SSD1306
Adafruit_GFX
LinkedIn
Github

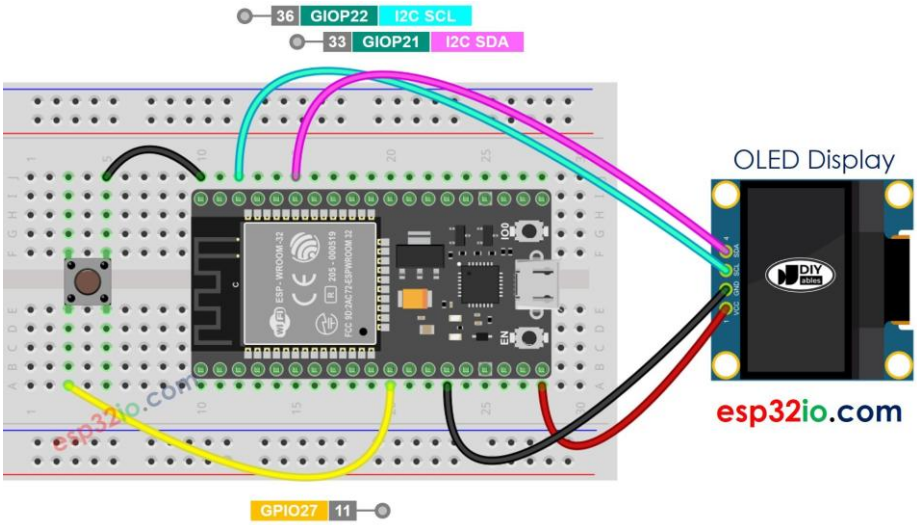
1. Overview

This collection of Arduino sketches explores what you can display on a tiny 128x64 OLED screen with an ESP32. All five folders share the same core setup **Adafruit_SSD1306** over I2C but each swaps in a different **PROGMEM bitmap array** to render something visually interesting, from falling snowflakes to popculture character portraits.

The key technique used across all bitmap projects is **image2cpp** from *image2cpp.com* a web tool that converts any image into a C byte array you can paste directly into Arduino code. The array is stored in flash memory using the **PROGMEM** keyword so it doesn't eat into the ESP32's RAM.

2. Hardware & Wiring

All projects use the same wiring. The ESP32 communicates with the OLED over I2C using its default pins:



ESP32 + SSD1306 OLED wiring (I2C). Button on GPIO27 is used in the button controlled variant

Pin Wiring between OLED(*SSD1306*) and ESP32 is :-

OLED Pin	ESP32 Pin
VCC	3.3V
GND	GND
SDA	GPIO 21
SCL	GPIO 22

3. Adafruit Demo (ssd1306_128x64_i2c)

This is the Adafruit library's builtin demo sketch, slightly adapted for the ESP32. It runs a complete tour of the display's drawing API, making it the ideal **first sketch** to flash when you get a new OLED and if everything renders correctly, your wiring and I2C address are good.

What it demos

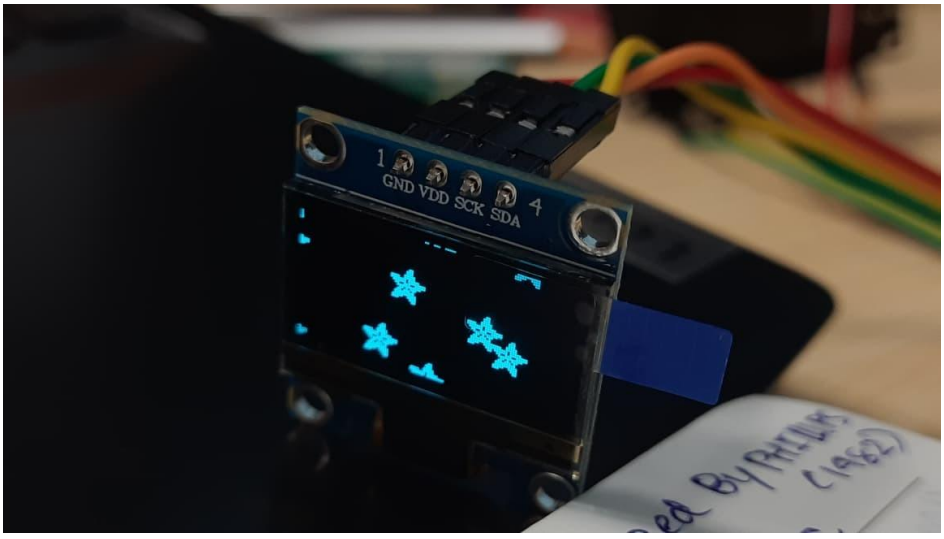
- **testdrawline()** - Animated lines sweeping from all four corners
- **testdrawrect() / testfillrect()** - Nested outlines and alternating filled rectangles
- **testdrawcircle() / testfillcircle()** - Expanding concentric circles, filled with invert effect
- **testdrawroundrect() / testfillroundrect()** - Rounded corners at quarterheight radius
- **testdrawtriangle() / testfilltriangle()** - Expanding triangles from center
- **testdrawchar()** - Full 256character Code Page 437 font dump
- **testdrawstyles()** - Normal, inverse, and 2xscale text
- **testscrolltext()** - Hardware horizontal and diagonal scroll commands
- **testdrawbitmap()** - Centered 16x16 logo drawn from a PROGMEM array
- **testanimate()** - 10 snowflakes falling with random speed (infinite loop)

The Snowflake Bitmap

The falling icon is a 16x16 pixel sprite stored as a PROGMEM byte array. Each row is two bytes (16 bits = 16 pixels). A 1 bit means white pixel, 0 means off:

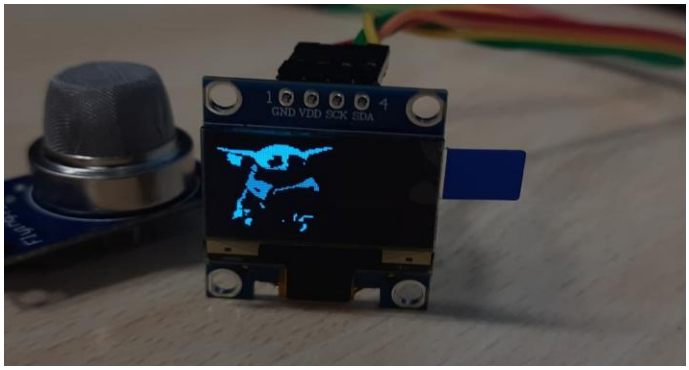
```
static const unsigned char PROGMEM logo_bmp[] = {
0b00000000, 0b11000000,  // Row 0
  0b00000001, 0b11000000,  // Row 1   // ... 14 more rows
...   0b00000000, 0b00110000   // Row 15
};
```

The **testanimate()** function initialises 10 snowflake positions with random X and a random vertical speed (1–5 px/frame). Each frame clears the buffer, redraws all flakes, calls **display.display()**, waits 200ms, then updates Y positions. Flakes that fall off the bottom are recycled back to the top.



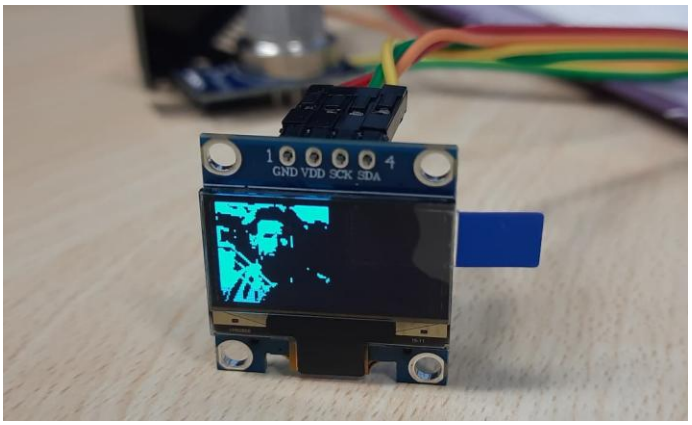
Snowflake animation running on the actual OLED hardware

This sketch displays a full 128x64 pixel bitmap of **Grogu** (Baby Yoda) as a static image. It demonstrates the simplest possible use of **drawBitmap()** to convert an image, paste the array, display it.

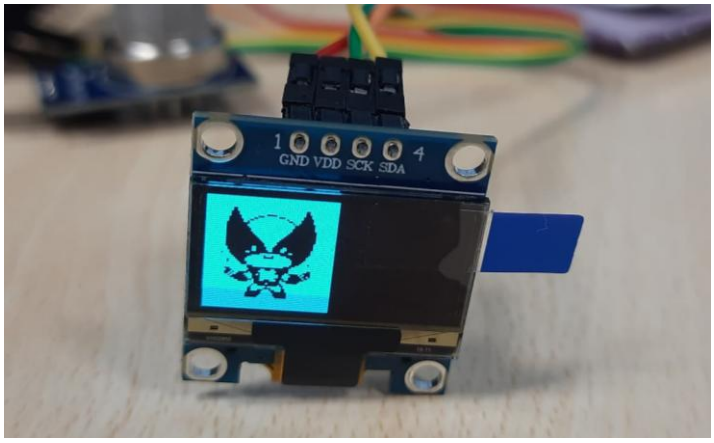


Grogu bitmap rendered on the 128x64 OLED

The same technique is reused for other images , with just the bitmap array swapped out.



Hugh Jackman bitmap rendered on the 128x64 OLED



Hugh Jackman bitmap rendered on the 128x64 OLED

How the bitmap rendering works

The entire sketch logic is just two calls after init:

```
95    // drawBitmap(x_coordinate, y_coordinate, bitmap_array, width, height, color)
96    display.drawBitmap(0, 0, my_image_bitmap, SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
97
98    display.display(); // Render the buffer onto the actual screen
99  }
```

The image array itself is 128 x 64 / 8 = **1024 bytes**. Each byte holds 8 horizontal pixels, MSB first. The **PROGMEM** qualifier stores this in flash rather than RAM — critical on microcontrollers where RAM is scarce.

How to make your own bitmap

- Go to **https://image2cpp.com**
- Upload any image (PNG, JPG, etc.)
- Set canvas size to **128 x 64**, background to **black**
- Set scaling to **fit** and brightness threshold to taste
- Choose output format: **Arduino**, draw mode: **Horizontal – 1 bit per pixel**
- Copy the generated array and paste it into your sketch in place of **my_image_bitmap**

4. Key Code Concepts

4.1 Initialisation (common to all sketches)

```
1   #include <Wire.h>
2   #include <Adafruit_GFX.h>
3   #include <Adafruit_SSD1306.h>
4
5   #define SCREEN_WIDTH 128 // OLED display width, in pixels
6   #define SCREEN_HEIGHT 64 // OLED display height, in pixels
7
8   // Declaration for an SSD1306 display connected to I2C (SDA, SCL pins)
9   #define OLED_RESET     -1 // Reset pin # (or -1 if sharing Arduino reset pin)
10  #define SCREEN_ADDRESS 0x3C /// See datasheet for Address; 0x3D or 0x3C
11
12  Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

4.2 PROGMEM & why it matters

A 128x64 bitmap is 1024 bytes. The ESP32 has plenty of flash (4MB+) but Arduinostyle global arrays default to RAM. **PROGMEM** forces the compiler to keep the data in flash and the library reads it with **pgm_read_byte()** internally:

```
// Correct — stored in flash
const unsigned char my_bitmap[] PROGMEM = { 0x00, 0xFF, ... };

// Wrong for large arrays — eats RAM
const unsigned char my_bitmap[] = { 0x00, 0xFF, ... };
```

4.3 display.display() - the flush call

All drawing functions (drawLine, drawBitmap, etc.) write to an **inmemory buffer**. Nothing appears on screen until you call **display.display()**, which transfers the 1024byte buffer to the OLED over I2C. Forgetting this call is the most common beginner mistake.

5. Library Setup

Install these two libraries via Arduino IDE -> Sketch -> Include Library -> Manage Libraries:

Library	Author	Install name
Adafruit SSD1306	Adafruit	Adafruit SSD1306
Adafruit GFX	Adafruit	Adafruit GFX Library

Also ensure your ESP32 board is installed: File -> Preferences -> add **https://dl.espressif.com/dl/package_esp32_index.json** to Additional Board URLs, then Tools -> Board -> Boards Manager -> search **esp32** -> install.

6. Summary

These five cover the two fundamental OLED display use cases on a microcontroller:

- **Graphics API demo (ssd1306_128x64_i2c)** which is great reference for every drawing primitive the library supports
- **Static bitmap rendering (Baby_Yoda, Hugh_Jackman, Wolverine)** the pattern for any custom graphic: convert -> PROGMEM array -> drawBitmap -> display

From here you can combine the two: animate a bitmap, overlay text on an image, or scroll through multiple portraits using a button, the wiring diagram already shows a button on GPIO27 for exactly that kind of interactive next step.
